

Socket Programming Practice Assignments
PCCSL507 – Computer Networks Laboratory
Department of Computer Science & Engineering

Submitted by:

Rose Soyuse

S5 CSE A

Roll number: 54

Socket Programming Practice Assignments

Question 1

Hello Client-Server (Beginner) Problem Statement Develop a TCP client-server application where:

- The client sends the message "Hello Server".
- The server receives the message and displays it.
- The server replies with "Hello Client".
- The client displays the received response.

Skills Practiced

- socket()
- bind()
- listen()
- accept()
- connect()
- send()
- recv()

Answer:

Client code:

```
C client.c > ...
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <winsock2.h>
5  #include <ws2tcpip.h>
6
7  #define PORT 8080
8
9  int main()
10 {
11     WSADATA wsa;
12     SOCKET sockfd;
13     struct sockaddr_in serverAddr;
14     char buffer[1024];
15
16     // Initialize Winsock
17     if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
18     {
19         printf("WSAStartup Failed\n");
20         return 1;
21     }
22
23     // Create Socket
24     sockfd = socket(AF_INET, SOCK_STREAM, 0);
25
26     if (sockfd == INVALID_SOCKET)
27     {
28         printf("Socket Creation Failed\n");
29         WSACleanup();
```

```

30     return 1;
31 }
32
33 printf("Socket Created Successfully...\n");
34
35 // Server Address
36 serverAddr.sin_family = AF_INET;
37 serverAddr.sin_port = htons(PORT);
38
39 inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);
40
41 // Connect
42 if (connect(sockfd,
43     (struct sockaddr *)&serverAddr,
44     sizeof(serverAddr)) == SOCKET_ERROR)
45 {
46     printf("Connection Failed\n");
47     closesocket(sockfd);
48     WSACleanup();
49     return 1;
50 }
51
52 printf("Connected to Server...\n");
53
54 // Send message
55 strcpy(buffer, "Hello Server");

```

```

    strcpy(buffer, "Hello Server");

    send(sockfd, buffer, strlen(buffer) + 1, 0);

    printf("Message Sent : %s\n", buffer);

    // Receive reply
    recv(sockfd, buffer, sizeof(buffer), 0);

    printf("Reply from Server : %s\n", buffer);

    // Close socket
    closesocket(sockfd);

    printf("Connection Closed...\n");

    WSACleanup();

    return 0;
}

```

Server code:

```
C server.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <winsock2.h>
5
6  #define PORT 8080
7
8  int main()
9  {
10     WSADATA wsa;
11     SOCKET serverSocket, clientSocket;
12     struct sockaddr_in serverAddr;
13     char buffer[1024];
14
15     // Initialize Winsock
16     if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
17     {
18         printf("WSAStartup Failed\n");
19         return 1;
20     }
21
22     // Create Socket
23     serverSocket = socket(AF_INET, SOCK_STREAM, 0);
24
25     if (serverSocket == INVALID_SOCKET)
26     {
27         printf("Socket Creation Failed\n");
28         WSACleanup();
29         return 1;
```

```
30     }
31
32     printf("Socket Created Successfully...\n");
33
34     // Server Address
35     serverAddr.sin_family = AF_INET;
36     serverAddr.sin_addr.s_addr = INADDR_ANY;
37     serverAddr.sin_port = htons(PORT);
38
39     // Bind
40     if (bind(serverSocket, (struct sockaddr *)&serverAddr,
41             sizeof(serverAddr)) == SOCKET_ERROR)
42     {
43         printf("Bind Failed\n");
44         closesocket(serverSocket);
45         WSACleanup();
46         return 1;
47     }
48
49     printf("Bind Successful...\n");
50
51     // Listen
52     listen(serverSocket, 5);
53
54     printf("Waiting for Client...\n");
55
56     // Accept
57     clientSocket = accept(serverSocket, NULL, NULL);
58
```

```

59     if (clientSocket == INVALID_SOCKET)
60     {
61         printf("Accept Failed\n");
62         closesocket(serverSocket);
63         WSACleanup();
64         return 1;
65     }
66     printf("Client Connected...\n");
67
68     // Receive message
69     recv(clientSocket, buffer, sizeof(buffer), 0);
70
71     printf("Client Says : %s\n", buffer);
72     // Send reply
73     strcpy(buffer, "Hello Client");
74
75     send(clientSocket, buffer, strlen(buffer) + 1, 0);
76
77     printf("Reply Sent...\n");
78     // Close sockets
79     closesocket(clientSocket);
80     closesocket(serverSocket);
81
82     printf("Connection Closed...\n");
83     WSACleanup();
84     return 0;
85 }

```

Output server

```

C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>gcc server.c -o server -lws2_
32

C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>server
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Client Says : Hello Server
Reply Sent...
Connection Closed...

C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>

```

Output Client

```

C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>gcc client.c -o client -lws2_
32

C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>client
Socket Created Successfully...
Connected to Server...
Message Sent : Hello Server
Reply from Server : Hello Client
Connection Closed...

C:\Users\Soyuse Peter\OneDrive\Desktop\CN_Lab>

```

Question 2

Arithmetic Server Problem Statement Develop a TCP client-server application where:

- The client inputs two integers.
- The client sends both numbers to the server.
- The server calculates:– Addition – Subtraction – Multiplication – Division
- The server sends all results back.
- The client displays the received results.

Skills Practiced

- Integer Communication
- Arithmetic Operations
- TCP Socket Programming

Answer:

Server code

```
C server.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <winsock2.h>
4
5  #define PORT 8080
6
7  int main()
8  {
9      WSADATA wsa;
10     SOCKET serverSocket, clientSocket;
11     struct sockaddr_in serverAddr;
12
13     int num1, num2;
14     int addition, subtraction, multiplication;
15     float division;
16
17     // Initialize Winsock
18     if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
19     {
20         printf("WSAStartup Failed\n");
21         return 1;
22     }
23
24     // Create Socket
25     serverSocket = socket(AF_INET, SOCK_STREAM, 0);
26
27     if (serverSocket == INVALID_SOCKET)
28     {
29         printf("Socket Creation Failed\n");
```

```

C server.c > main()
27     if (serverSocket == INVALID_SOCKET)
28     {
29         printf("Socket Creation Failed\n");
30         WSACleanup();
31         return 1;
32     }
33
34     printf("Socket Created Successfully...\n");
35
36     // Server Address
37     serverAddr.sin_family = AF_INET;
38     serverAddr.sin_addr.s_addr = INADDR_ANY;
39     serverAddr.sin_port = htons(PORT);
40
41     // Bind
42     if (bind(serverSocket,
43             (struct sockaddr *)&serverAddr,
44             sizeof(serverAddr)) == SOCKET_ERROR)
45     {
46         printf("Bind Failed\n");
47         closesocket(serverSocket);
48         WSACleanup();
49         return 1;
50     }
51
52     printf("Bind Successful...\n");
53
54     // Listen
55     listen(serverSocket, 5);
56
57     printf("Waiting for Client...\n");
58
59     // Accept
60     clientSocket = accept(serverSocket, NULL, NULL);
61
62     if (clientSocket == INVALID_SOCKET)
63     {
64         printf("Accept Failed\n");
65         closesocket(serverSocket);
66         WSACleanup();
67         return 1;
68     }
69
70     printf("Client Connected...\n");
71
72     // Receive numbers
73     recv(clientSocket, (char *)&num1, sizeof(num1), 0);
74     recv(clientSocket, (char *)&num2, sizeof(num2), 0);
75
76     printf("Received Numbers : %d and %d\n", num1, num2);
77
78     // Perform calculations
79     addition = num1 + num2;
80     subtraction = num1 - num2;
81     multiplication = num1 * num2;

```

```

82
83     if (num2 != 0)
84         division = (float)num1 / num2;
85     else
86         division = 0;
87
88     // Send results
89     send(clientSocket, (char *)&addition, sizeof(addition), 0);
90     send(clientSocket, (char *)&subtraction, sizeof(subtraction), 0);
91     send(clientSocket, (char *)&multiplication, sizeof(multiplication), 0);
92     send(clientSocket, (char *)&division, sizeof(division), 0);
93
94     printf("Results Sent Successfully...\n");
95
96     // Close
97     closesocket(clientSocket);
98     closesocket(serverSocket);
99
100    printf("Connection Closed...\n");
101
102    WSACleanup();
103
104    return 0;
105 }

```

Client code:

```

C clientc > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <winsock2.h>
4
5  #define PORT 8080
6
7  int main()
8  {
9      WSADATA wsa;
10     SOCKET sockfd;
11     struct sockaddr_in serverAddr;
12
13     int num1, num2;
14     int addition, subtraction, multiplication;
15     float division;
16
17     // Initialize Winsock
18     if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
19     {
20         printf("WSAStartup Failed\n");
21         return 1;
22     }
23
24     // Create Socket
25     sockfd = socket(AF_INET, SOCK_STREAM, 0);
26
27     if (sockfd == INVALID_SOCKET)
28     {
29         printf("Socket Creation Failed\n");

```



```

29     printf("Socket Creation Failed\n");
30     WSACleanup();
31     return 1;
32 }
33
34 printf("Socket Created Successfully...\n");
35
36
37
38 serverAddr.sin_port = htons(PORT);
39 serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
40
41 // Connect
42 if (connect(sockfd,
43     (struct sockaddr *)&serverAddr,
44     sizeof(serverAddr)) == SOCKET_ERROR)
45 {
46     printf("Connection Failed\n");
47     closesocket(sockfd);
48     WSACleanup();
49     return 1;
50 }
51
52 printf("Connected to Server...\n");
53
54 // Input numbers
55 printf("Enter first integer : ");
56 scanf("%d", &num1);
57

```

```

58     printf("Enter second integer : ");
59     scanf("%d", &num2);
60
61 // Send numbers
62 send(sockfd, (char *)&num1, sizeof(num1), 0);
63 send(sockfd, (char *)&num2, sizeof(num2), 0);
64
65 printf("Numbers Sent Successfully...\n");
66
67 // Receive results
68 recv(sockfd, (char *)&addition, sizeof(addition), 0);
69 recv(sockfd, (char *)&subtraction, sizeof(subtraction), 0);
70 recv(sockfd, (char *)&multiplication, sizeof(multiplication), 0);
71 recv(sockfd, (char *)&division, sizeof(division), 0);
72
73 // Display results
74 printf("\nResults from Server\n");
75 printf("Addition      : %d\n", addition);
76 printf("Subtraction    : %d\n", subtraction);
77 printf("Multiplication  : %d\n", multiplication);
78 printf("Division       : %.2f\n", division);
79
80 // Close
81 closesocket(sockfd);
82 printf("\nConnection Closed...\n");
83 WSACleanup();
84 return 0;
85 }

```

Server output

```
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Numbers : 20 and 5
Results Sent Successfully...
Connection Closed...

C:\Users\Soyuse Peter\OneDrive\Desktop\Q2_Arithmetic>
```

Client output

```
Socket Created Successfully...
Connected to Server...
Enter first integer : 20
Enter second integer : 5
Numbers Sent Successfully...

Results from Server
Addition      : 25
Subtraction   : 15
Multiplication : 100
Division      : 4.00

Connection Closed...
```

Question 3

String Processing Server Problem Statement Develop a TCP client-server application where:

- The client inputs a string.
- The server performs the following operations: – Find the string length – Convert the string to uppercase – Reverse the string
- The server sends the processed results.
- The client displays all outputs.

Skills Practiced • Character Arrays • String Manipulation • TCP Communication

Answer:

Server code

```
C server.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <ctype.h>
5  #include <winsock2.h>
6
7  #define PORT 8080
8
9  int main()
10 {
11     WSADATA wsa;
12     SOCKET serverSocket, clientSocket;
13     struct sockaddr_in serverAddr;
14
15     char buffer[1024];
16     char uppercase[1024];
17     char reversed[1024];
18     int length;
19
20     if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
21     {
22         printf("WSAStartup Failed\n");
23         return 1;
24     }
25
26     serverSocket = socket(AF_INET, SOCK_STREAM, 0);
27
28     if (serverSocket == INVALID_SOCKET)
29     {
```

```
30         printf("Socket Creation Failed\n");
31         WSACleanup();
32         return 1;
33     }
34
35     printf("Socket Created Successfully...\n");
36
37     serverAddr.sin_family = AF_INET;
38     serverAddr.sin_addr.s_addr = INADDR_ANY;
39     serverAddr.sin_port = htons(PORT);
40
41     if (bind(serverSocket,
42             (struct sockaddr *)&serverAddr,
43             sizeof(serverAddr)) == SOCKET_ERROR)
44     {
45         printf("Bind Failed\n");
46         closesocket(serverSocket);
47         WSACleanup();
48         return 1;
49     }
50
51     printf("Bind Successful...\n");
52
53     listen(serverSocket, 5);
54
55     printf("Waiting for Client...\n");
56
57     clientSocket = accept(serverSocket, NULL, NULL);
```

```
59     if (clientSocket == INVALID_SOCKET)
60     {
61         printf("Accept Failed\n");
62         closesocket(serverSocket);
63         WSACleanup();
64         return 1;
65     }
66
67     printf("Client Connected...\n");
68
69     recv(clientSocket, buffer, sizeof(buffer), 0);
70
71     printf("Received String : %s\n", buffer);
72
73     // Find length
74     length = strlen(buffer);
75
76     // Convert to uppercase
77     strcpy(uppercase, buffer);
78
79     for (int i = 0; uppercase[i] != '\0'; i++)
80     {
81         uppercase[i] = toupper(uppercase[i]);
82     }
83
84     // Reverse string
85     for (int i = 0; i < length; i++)
86     {
87         reversed[i] = buffer[length - 1 - i];
88     }
89
90     reversed[length] = '\0';
91
92     // Send results
93     send(clientSocket, (char *)&length, sizeof(length), 0);
94     send(clientSocket, uppercase, strlen(uppercase) + 1, 0);
95     send(clientSocket, reversed, strlen(reversed) + 1, 0);
96
97     printf("Processed Results Sent Successfully...\n");
98
99     closesocket(clientSocket);
100    closesocket(serverSocket);
101
102    printf("Connection Closed...\n");
103
104    WSACleanup();
105
106    return 0;
107 }
```

Client code:

```
C client.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <winsock2.h>
5
6  #define PORT 8080
7
8  int main()
9  {
10     WSADATA wsa;
11     SOCKET sockfd;
12     struct sockaddr_in serverAddr;
13
14     char buffer[1024];
15     int length;
16
17     if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
18     {
19         printf("WSAStartup Failed\n");
20         return 1;
21     }
22
23     sockfd = socket(AF_INET, SOCK_STREAM, 0);
24
25     if (sockfd == INVALID_SOCKET)
26     {
27         printf("Socket Creation Failed\n");
28         WSACleanup();
29         return 1;
30     }
31
32     printf("Socket Created Successfully...\n");
33
34     serverAddr.sin_family = AF_INET;
35     serverAddr.sin_port = htons(PORT);
36     serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
37
38     if (connect(sockfd,
39         (struct sockaddr *)&serverAddr,
40         sizeof(serverAddr)) == SOCKET_ERROR)
41     {
42         printf("Connection Failed\n");
43         closesocket(sockfd);
44         WSACleanup();
45         return 1;
46     }
47
48     printf("Connected to Server...\n");
49
50     printf("Enter a string : ");
51     fgets(buffer, sizeof(buffer), stdin);
52
53     // Remove newline
54     buffer[strcspn(buffer, "\n")] = '\0';
55
56     send(sockfd, buffer, strlen(buffer) + 1, 0);
57
58     printf("String Sent Successfully...\n");
```

```

60 // Receive results
61 recv(sockfd, (char *)&length, sizeof(length), 0);
62
63 recv(sockfd, buffer, sizeof(buffer), 0);
64
65 char uppercase[1024];
66 strcpy(uppercase, buffer);
67
68 recv(sockfd, buffer, sizeof(buffer), 0);
69
70 char reversed[1024];
71 strcpy(reversed, buffer);
72
73 printf("\nResults from Server\n");
74 printf("String Length : %d\n", length);
75 printf("Uppercase      : %s\n", uppercase);
76 printf("Reversed       : %s\n", reversed);
77
78 closesocket(sockfd);
79
80 printf("\nConnection Closed...\n");
81
82 WSACleanup();
83
84 return 0;
85 }

```

Server output:

```

C:\Users\Soyuse Peter\OneDrive\Desktop\Q2_Arithmetic>gcc server.c -o server
-lws2_32

C:\Users\Soyuse Peter\OneDrive\Desktop\Q2_Arithmetic>server
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received String : Hello World
Processed Results Sent Successfully...
Connection Closed...

```

Client output:

```

C:\Users\Soyuse Peter\OneDrive\Desktop\Q2_Arithmetic>client
Socket Created Successfully...
Connected to Server...
Enter a string : Hello World
String Sent Successfully...

Results from Server
String Length : 11
Uppercase      : HELLO WORLD
Reversed       : dlroW olleH

Connection Closed...

```

Question 4

Array Processing Server Problem Statement Develop a TCP client-server application where:

- The client inputs N integers.
- The array is sent to the server.
- The server computes: – Sum – Average – Maximum Element – Minimum Element
- The server sends the results.
- The client displays them.

Skills Practiced • Arrays • Loops • TCP Socket Programming

Answer:

Server code:

```
C server.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <winsock2.h>
4
5  #define PORT 8080
6  #define MAX 100
7
8  int main()
9  {
10     WSADATA wsa;
11     SOCKET serverSocket, clientSocket;
12     struct sockaddr_in serverAddr;
13
14     int n;
15     int arr[MAX];
16     int sum = 0;
17     int maximum, minimum;
18     float average;
19
20     // Initialize Winsock
21     if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
22     {
23         printf("WSAStartup Failed\n");
24         return 1;
25     }
26
27     // Create Socket
28     serverSocket = socket(AF_INET, SOCK_STREAM, 0);
29
```

```

30     if (serverSocket == INVALID_SOCKET)
31     {
32         printf("Socket Creation Failed\n");
33         WSACleanup();
34         return 1;
35     }
36
37     printf("Socket Created Successfully...\n");
38
39     // Server Address
40     serverAddr.sin_family = AF_INET;
41     serverAddr.sin_addr.s_addr = INADDR_ANY;
42     serverAddr.sin_port = htons(PORT);
43
44     // Bind
45     if (bind(serverSocket,
46             (struct sockaddr *)&serverAddr,
47             sizeof(serverAddr)) == SOCKET_ERROR)
48     {
49         printf("Bind Failed\n");
50         closesocket(serverSocket);
51         WSACleanup();
52         return 1;
53     }
54
55     printf("Bind Successful...\n");
56
57     // Listen
58     listen(serverSocket, 5);
59
60     printf("Waiting for Client...\n");
61
62     // Accept
63     clientSocket = accept(serverSocket, NULL, NULL);
64
65     if (clientSocket == INVALID_SOCKET)
66     {
67         printf("Accept Failed\n");
68         closesocket(serverSocket);
69         WSACleanup();
70         return 1;
71     }
72
73     printf("Client Connected...\n");
74
75     // Receive N
76     recv(clientSocket, (char *)&n, sizeof(n), 0);
77
78     // Receive array
79     recv(clientSocket, (char *)arr, sizeof(int) * n, 0);
80
81     printf("Received Array : ");
82
83     for (int i = 0; i < n; i++)
84     {
85         printf("%d ", arr[i]);
86     }
87
88     printf("\n");

```



```

90     // Calculate sum
91     for (int i = 0; i < n; i++)
92     {
93         sum += arr[i];
94     }
95
96     // Find maximum and minimum
97     maximum = arr[0];
98     minimum = arr[0];
99
100    for (int i = 1; i < n; i++)
101    {
102        if (arr[i] > maximum)
103            maximum = arr[i];
104
105        if (arr[i] < minimum)
106            minimum = arr[i];
107    }
108
109    average = (float)sum / n;
110
111    // Send results
112    send(clientSocket, (char *)&sum, sizeof(sum), 0);
113    send(clientSocket, (char *)&average, sizeof(average), 0);
114    send(clientSocket, (char *)&maximum, sizeof(maximum), 0);
115    send(clientSocket, (char *)&minimum, sizeof(minimum), 0);
116
117    printf("Results Sent Successfully...\n");
118

```

```

119    // Close
120    closesocket(clientSocket);
121    closesocket(serverSocket);
122
123    printf("Connection Closed...\n");
124
125    WSACleanup();
126
127    return 0;
128 }

```

Server output:

```

Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Array : 5 10 20 30 40 50
Results Sent Successfully...
Connection Closed...

```

Client output:

```
Socket Created Successfully...
Connected to Server...
Enter number of elements : 6
Enter 6 integers:
5
10
20
30
40
50
Array Sent Successfully...

Results from Server
Sum      : 155
```

```
Results from Server
Sum      : 155
Average  : 25.83
Maximum  : 50
Minimum  : 5

Connection Closed...
```

Question 5 – Matrix Analyzer Problem Statement Develop a TCP client-server application where:

- The client inputs a square matrix of order N.
- The matrix is sent to the server.
- The server determines: – Matrix Type – Sum of Matrix Elements – Trace of Matrix
- The server sends the results.
- The client displays the received information.

Matrix Types • Diagonal Matrix • Upper Triangular Matrix • Lower Triangular Matrix • General Matrix Skills Practiced • Two-Dimensional Arrays • Matrix Operations • TCP Socket Programming

Answer:

Server code:

```

C server.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <winsock2.h>
4
5  #define PORT 8080
6  #define MAX 100
7
8  int main()
9  {
10     WSADATA wsa;
11     SOCKET serverSocket, clientSocket;
12     struct sockaddr_in serverAddr;
13
14     int n;
15     int matrix[MAX][MAX];
16
17     int sum = 0;
18     int trace = 0;
19
20     int diagonal = 1;
21     int upper = 1;
22     int lower = 1;
23
24     char result[100];
25
26     // Initialize Winsock
27     if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
28     {
29         printf("WSAStartup Failed\n");

```

```

30     return 1;
31 }
32
33 // Create Socket
34 serverSocket = socket(AF_INET, SOCK_STREAM, 0);
35
36 if (serverSocket == INVALID_SOCKET)
37 {
38     printf("Socket Creation Failed\n");
39     WSACleanup();
40     return 1;
41 }
42
43 printf("Socket Created Successfully...\n");
44
45 // Server Address
46 serverAddr.sin_family = AF_INET;
47 serverAddr.sin_addr.s_addr = INADDR_ANY;
48 serverAddr.sin_port = htons(PORT);
49
50 // Bind
51 if (bind(serverSocket,
52         (struct sockaddr *)&serverAddr,
53         sizeof(serverAddr)) == SOCKET_ERROR)
54 {
55     printf("Bind Failed\n");
56     closesocket(serverSocket);
57     WSACleanup();
58     return 1;

```

```

59     }
60
61     printf("Bind Successful...\n");
62
63     // Listen
64     listen(serverSocket, 5);
65
66     printf("Waiting for Client...\n");
67
68     // Accept
69     clientSocket = accept(serverSocket, NULL, NULL);
70
71     if (clientSocket == INVALID_SOCKET)
72     {
73         printf("Accept Failed\n");
74         closesocket(serverSocket);
75         WSACleanup();
76         return 1;
77     }
78
79     printf("Client Connected...\n");
80
81     // Receive matrix order
82     recv(clientSocket, (char *)&n, sizeof(n), 0);
83
84     // Receive matrix
85     recv(clientSocket,
86          (char *)matrix,
87          sizeof(int) * n * n,

```

```

88          0);
89
90     printf("Received Matrix:\n");
91
92     for (int i = 0; i < n; i++)
93     {
94         for (int j = 0; j < n; j++)
95         {
96             printf("%d ", matrix[i][j]);
97         }
98         printf("\n");
99     }
100
101     // Analyze matrix
102     for (int i = 0; i < n; i++)
103     {
104         for (int j = 0; j < n; j++)
105         {
106             sum += matrix[i][j];
107
108             if (i == j)
109                 trace += matrix[i][j];
110
111             // For diagonal matrix
112             if (i != j && matrix[i][j] != 0)
113                 diagonal = 0;
114
115             // For upper triangular matrix
116             if (i > j && matrix[i][j] != 0)

```

```

116         if (i > j && matrix[i][j] != 0)
117             upper = 0;
118
119         // For lower triangular matrix
120         if (i < j && matrix[i][j] != 0)
121             lower = 0;
122     }
123 }
124
125 // Determine matrix type
126 if (diagonal)
127     strcpy(result, "Diagonal Matrix");
128 else if (upper)
129     strcpy(result, "Upper Triangular Matrix");
130 else if (lower)
131     strcpy(result, "Lower Triangular Matrix");
132 else
133     strcpy(result, "General Matrix");
134
135 // Send results
136 send(clientSocket,
137      result,
138      strlen(result) + 1,
139      0);
140
141 send(clientSocket,
142      (char *)&sum,
143      sizeof(sum),
144      0);

```

```

145
146 send(clientSocket,
147      (char *)&trace,
148      sizeof(trace),
149      0);
150
151 printf("Matrix Type : %s\n", result);
152 printf("Sum : %d\n", sum);
153 printf("Trace : %d\n", trace);
154
155 printf("Results Sent Successfully...\n");
156
157 // Close
158 closesocket(clientSocket);
159 closesocket(serverSocket);
160
161 printf("Connection Closed...\n");
162
163 WSACleanup();
164
165 return 0;
166 }

```

Client code:

```
C client.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <winsock2.h>
4
5  #define PORT 8080
6  #define MAX 100
7
8  int main()
9  {
10     WSADATA wsa;
11     SOCKET sockfd;
12     struct sockaddr_in serverAddr;
13
14     int n;
15     int matrix[MAX][MAX];
16
17     char result[100];
18
19     int sum;
20     int trace;
21
22     // Initialize Winsock
23     if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
24     {
25         printf("WSAStartup Failed\n");
26         return 1;
27     }
28
29     // Create Socket
30     sockfd = socket(AF_INET, SOCK_STREAM, 0);
31
32     if (sockfd == INVALID_SOCKET)
33     {
34         printf("Socket Creation Failed\n");
35         WSACleanup();
36         return 1;
37     }
38
39     printf("Socket Created Successfully...\n");
40
41     // Server Address
42     serverAddr.sin_family = AF_INET;
43     serverAddr.sin_port = htons(PORT);
44     serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
45
46     // Connect
47     if (connect(sockfd,
48         (struct sockaddr *)&serverAddr,
49         sizeof(serverAddr)) == SOCKET_ERROR)
50     {
51         printf("Connection Failed\n");
52         closesocket(sockfd);
53         WSACleanup();
54         return 1;
55     }
56
57     printf("Connected to Server...\n");
58
```

```

59 // Input matrix order
60 printf("Enter Matrix Order : ");
61 scanf("%d", &n);
62
63 // Input matrix
64 printf("Enter Matrix Elements:\n");
65
66 for (int i = 0; i < n; i++)
67 {
68     for (int j = 0; j < n; j++)
69     {
70         scanf("%d", &matrix[i][j]);
71     }
72 }
73
74 // Display matrix
75 printf("\nMatrix:\n");
76
77 for (int i = 0; i < n; i++)
78 {
79     for (int j = 0; j < n; j++)
80     {
81         printf("%d ", matrix[i][j]);
82     }
83     printf("\n");
84 }
85
86 // Send matrix order
87 send(sockfd,

```

```

88     (char *)&n,
89     sizeof(n),
90     0);
91
92 // Send matrix
93 send(sockfd,
94     (char *)matrix,
95     sizeof(int) * n * n,
96     0);
97
98 printf("\nMatrix Sent Successfully...\n");
99
100 // Receive results
101 recv(sockfd,
102     result,
103     sizeof(result),
104     0);
105
106 recv(sockfd,
107     (char *)&sum,
108     sizeof(sum),
109     0);
110
111 recv(sockfd,
112     (char *)&trace,
113     sizeof(trace),
114     0);
115
116 // Display results

```

```

116     // Display results
117     printf("\nResults from Server\n");
118     printf("Matrix Type : %s\n", result);
119     printf("Sum          : %d\n", sum);
120     printf("Trace         : %d\n", trace);
121
122     // Close
123     closesocket(sockfd);
124
125     printf("\nConnection Closed...\n");
126
127     WSACleanup();
128
129     return 0;
130 }

```

Server output:

```

Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Matrix:
1 0 0
0 0 0
0 0 0
Matrix Type : Diagonal Matrix
Sum : 1
Trace : 1
Results Sent Successfully...
Connection Closed...

```

Client output:

```

Socket Created Successfully...
Connected to Server...
Enter Matrix Order : 3
Enter Matrix Elements:
1 0 0 0 2 0 0 0 3

Matrix:
1 0 0
0 2 0
0 0 3

Matrix Sent Successfully...

Results from Server
Matrix Type : Diagonal Matrix
Sum          : 1
Trace        : 1

Connection Closed...

```